Cloud Email Scheduler

Functionality-first implementation plan • Personal Gmail • Up to 200 emails per batch

Purpose: build a private, cloud-hosted tool that imports CSV/JSON batches, lets you review unique email addresses, subjects and plain-text bodies, attaches one common CV PDF, and sends on a schedule while your laptop is off.

Recommended stack: Next.js + TypeScript on Vercel, Supabase PostgreSQL and private Storage, Gmail API with Google OAuth, and one persistent Node.js/TypeScript background worker hosted separately (for example, a suitable Render Background Worker plan).

Confirmed requirements

Setting	Decision
Sending account	One personal Gmail account using Gmail API / OAuth
Import	CSV and JSON
Per-recipient content	Different recipient, subject and plain-text body per row
Attachment	One shared CV PDF per batch
Scheduling	Start now or at a future date/time
Interval	Fixed configurable interval, 5–10 seconds requested
Batch size	Up to 200 emails
Controls	Pause, resume, cancel and skip
Failure handling	Automatic retries for clearly retryable errors
History	Retain batches and logs until manually deleted
Access	Single-user tool; manual confirmation before each batch

Key engineering constraint

The database-backed queue is the source of truth. Do not rely on a browser timer or a long-running Next.js request. A persistent cloud worker must claim due jobs, enforce scheduling, call Gmail, record results, and recover after restarts.

1. Architecture

Use a small monolithic dashboard plus a separate worker. This keeps the MVP simple while making scheduled sending independent of the laptop.

Component	Technology	Responsibilities
Dashboard and short API routes	Next.js + TypeScript on Vercel	Import files, validate and preview rows, upload CV, schedule batches, control jobs and show status.
Database	Supabase PostgreSQL	Persist batches, queue rows, schedule times, attempt history, statuses and counters.
Attachment storage	Supabase Storage private bucket	Store the shared CV PDF; worker downloads it through server-side access.
Sender	Gmail API + Google OAuth 2.0	Send each email separately and return Gmail message IDs.
Queue worker	Node.js + TypeScript on persistent worker hosting	Poll due jobs, claim one job atomically, send, retry, respect pause/cancel and recover after crashes.

Data flow

1. User uploads CSV/JSON and a CV PDF.
2. Server validates input and saves the batch as a draft in Supabase.
3. User reviews the exact recipients, subjects, bodies, attachment and schedule.
4. User confirms. The batch becomes scheduled or running and jobs are persisted.
5. The cloud worker claims one eligible job, checks controls and quotas, then calls Gmail.
6. The worker records success/failure and schedules the next eligible job.
7. The dashboard reads persisted statuses; closing the browser or laptop does not stop sending.

Hosting note

Vercel is appropriate for the dashboard and short API routes, but not for a process that must run indefinitely. Deploy the worker as a separate persistent background service. Confirm its plan supports always-on execution and automatic restart; hosting costs and provider policies can change.

2. Database schema

Use migrations and keep the schema minimal. Enable Row Level Security where applicable and expose no service-role credentials to the browser.

Table	Core fields	Purpose
email_batches	id, name, status, start_at, interval_seconds, total_count, sent_count, failed_count, skipped_count, created_at, completed_at	One row per batch; controls batch lifecycle and summary.
scheduled_emails	id, batch_id, recipient, subject, body_text, status, scheduled_at, attempt_count, next_attempt_at, lease_until, provider_message_id, last_error, sent_at, idempotency_key	One persistent job per recipient.
email_attempts	id, email_id, attempt_number, started_at, finished_at, outcome, error_code, provider_message_id	Append-only audit of attempts and outcomes.
connected_accounts	id, email_address, encrypted_refresh_token, token_expiry, connection_status	Minimal record for the one Gmail account; protect secrets.

Indexes and constraints

- Index scheduled_emails on (status, scheduled_at) and (batch_id, status).
- Use a unique idempotency key for each queued job to prevent accidental duplicate job creation.

- Use atomic database claiming (transaction/RPC) and a lease timestamp so a restarted worker can recover stale claims.
- Keep batch counters derived from or reconciled against email statuses; do not rely on in-memory counters.
- Store UTC timestamps in the database and convert to the user's local timezone only in the interface.

Recommended statuses

Batch: draft, scheduled, running, paused, completed, cancelled, needs_attention. Email job: pending, sending, sent, retrying, failed, skipped, uncertain. An uncertain outcome means Gmail may have accepted the message but the worker did not record the response.

3. Import, preview and CV attachment

CSV/JSON input fields

Each record contains email, subject and body. CSV cells containing commas or newlines must be correctly quoted. JSON is an array of objects with the same keys.

Validation steps

1. Limit upload size and reject unsupported file extensions/content types.
2. Parse on the server and validate every row with Zod.
3. Trim surrounding whitespace; reject missing recipient, subject or body and malformed email syntax.
4. Detect duplicate recipients in a batch and report them for review rather than silently dropping rows.
5. Show row-level errors and a preview of all message contents before scheduling.
6. Save only after validation; require manual confirmation before the batch is activated.

CV storage

- Upload one PDF to a private Supabase Storage bucket and associate its object key with the batch.
- Restrict file size and MIME type; use server-side access to download the PDF when building each email.
- Do not expose a public URL or place the PDF in client-side persistent storage.
- Keep the file while queued/retryable jobs depend on it. Add deletion controls later if needed.
- Validate total MIME message size against Gmail/API constraints and test the attachment end-to-end.

4. Gmail OAuth and message sending

1. Create/select a Google Cloud project and enable the Gmail API.
2. Configure the OAuth consent screen, authorized redirect URL and required test/production settings.
3. Request the narrow sending scope: https://www.googleapis.com/auth/gmail.send, if sufficient for the implementation.
4. Use server-side OAuth authorization with offline access; securely store and refresh the token so the worker can run when the user is offline.
5. Implement one server-side sendEmail function that builds a MIME message with recipient, subject, plain-text body and PDF attachment.
6. Call Gmail users.messages.send and persist the returned Gmail message ID and timestamp.
7. Provide a reconnect flow if consent is revoked or the refresh token stops working.

OAuth caveat: Google consent-screen mode and scope verification requirements may restrict test users or production use. Set up OAuth early and verify the real deployment before building the whole queue around it.

5. Persistent worker and scheduling

Worker responsibilities

- Run as a persistent cloud process, with automatic restart and a health/heartbeat check.
- Poll or wait for the next due job; use PostgreSQL as the source of truth.
- Claim only one eligible job at a time for this single Gmail account.
- Before sending, verify batch status, job status, schedule, retry time, OAuth state and provider throttling.
- Record every attempt, classify the outcome and update persistent status.
- After a restart, recover expired leases and resume eligible jobs without resetting the batch.

Interval behavior

For interval g, the nominal target is t(n) = t(0) + n × g. Prefer enforcing the minimum gap from completion of the previous attempt before starting the next message, rather than overlapping requests to chase a wall-clock target. Provider rate limits and retry delays always override the selected interval.

Interval	200 messages: first to last target time
5 seconds	16 minutes 35 seconds
10 seconds	33 minutes 10 seconds

These are theoretical timings assuming the first email starts immediately. Network latency, Gmail throttling, backoff and retries extend total completion time. A 5–10 second gap does not bypass Gmail quotas or anti-abuse protections.

6. Queue controls and lifecycle

Action	Required behavior
Pause	Stop claiming new jobs after safely resolving the current attempt; persist paused status.
Resume	Continue eligible pending jobs from the database without losing schedule/history.
Cancel	Cancel jobs not yet started; preserve already-sent messages and attempt logs.
Skip	Mark one pending job skipped and proceed to the next eligible job.
Manual confirmation	Require explicit review and confirmation before a draft/scheduled batch is activated.

Make controls idempotent: repeating Pause, Resume, Skip or Cancel should not corrupt state or recreate jobs.

7. Retry, idempotency and crash recovery

Failure	Handling
Network failure before request	Retry with bounded exponential backoff.
HTTP 429 / rate limit	Honor Retry-After/provider guidance and slow or pause the queue.
HTTP 5xx	Retry with backoff and a maximum attempt count.
Invalid address / malformed message	Mark failed; do not automatically retry.
Refresh token revoked	Pause and require reconnection.
Timeout after possible acceptance	Mark uncertain and reconcile; do not blindly resend.
Worker crash	Recover expired lease and inspect prior attempt before retrying.

Exactly-once sending cannot be guaranteed by a database alone. Gmail's send endpoint does not provide a general idempotency key for message submission. A crash after Gmail accepts a message but before the

database records success can create ambiguity; model that state explicitly.

8. Security, quotas and deliverability

- Protect the dashboard with single-user authentication and enforce authorization on every server route.
- Keep Google client secrets, refresh-token encryption keys and Supabase service-role keys on the server only.
- Encrypt stored OAuth refresh tokens and provide a disconnect/revoke process.
- Use private storage and least-privilege database permissions; never expose secrets in client bundles or logs.
- Keep logs free of tokens and unnecessary message content; retain the history needed for debugging.
- Respect Gmail daily and API quotas, back off on rate limits, and stop the queue when authorization or account state needs attention.
- A Gmail acceptance response is not proof of inbox placement. Use relevant, individually tailored academic outreach and honor requests not to be contacted.

A personal Gmail account commonly has a published limit of 500 emails per day, but account-specific restrictions, API quotas and anti-abuse controls may apply. Verify current Google documentation during implementation rather than treating the daily number as a delivery guarantee.

9. Deployment plan

1. Create the repository and environment files; add linting, TypeScript checks and tests.
2. Create Supabase migrations, private storage bucket, indexes, constraints and secure queue-claiming RPC.
3. Build and test CSV/JSON parsing, validation, preview and draft persistence.
4. Set up Google Cloud OAuth and implement a test email with a CV attachment.
5. Build the worker locally using persisted jobs and a configurable interval.
6. Deploy the dashboard to Vercel and configure production redirect URLs and environment variables.
7. Deploy the worker to a persistent service (for example, a suitable Render Background Worker plan). Confirm always-on behavior, restart policy, environment variables and cost.
8. Run end-to-end tests with a small batch; only then use a 200-recipient batch.
9. Add worker health checks, basic error alerts and a database backup/restore check.

Environment variables (names illustrative)

Variable	Where used
NEXT_PUBLIC_SUPABASE_URL	Dashboard/browser client; public project URL only
NEXT_PUBLIC_SUPABASE_ANON_KEY	Browser client; rely on RLS and never treat as a secret
SUPABASE_SERVICE_ROLE_KEY	Server/worker only; never expose to browser
GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET	Server/worker OAuth
OAUTH_TOKEN_ENCRYPTION_KEY	Server/worker only
APP_BASE_URL / GOOGLE_REDIRECT_URI	OAuth callback configuration

10. Fastest build sequence and acceptance criteria

Phase	Build	Acceptance criteria
1. Import and data	Schema, CSV/JSON import, validation, preview, draft batch.	Valid rows persist; invalid rows are explained; no email is sent.
2. Gmail integration	OAuth, token refresh, MIME builder, CV attachment, test-send path.	A test message arrives with correct recipient, subject, body and attachment.
3. Queue worker	Atomic claim, interval enforcement, scheduling, statuses and basic retries.	Queue runs without browser open; restart preserves work.
4. Controls and recovery	Pause/resume/cancel/skip, uncertain outcomes, attempt history.	State transitions are safe and persisted; uncertain sends are not blindly retried.
5. Deployment and hardening	Vercel + persistent worker, secrets, security checks, monitoring.	Laptop can be off; worker health and queue progress are observable.

Time estimate

A rough target is 4–8 focused hours for a prototype and around 1–3 days for a hosted MVP with OAuth, worker reliability and testing. This is an estimate, not a guarantee; Google OAuth configuration, deployment and failure-case testing may take longer.

11. Pre-launch test checklist

- Import valid CSV and JSON; reject malformed rows and missing fields.
- Confirm each recipient receives the matching subject and body.
- Send and verify the shared PDF attachment.
- Schedule a future batch and confirm it does not start early.
- Turn off the laptop and confirm the cloud worker continues.
- Pause/resume, cancel, and skip jobs while the queue is active.
- Simulate 429, 5xx, network failure, token revocation and worker restart.
- Simulate an uncertain send outcome and verify no blind duplicate retry occurs.
- Verify that the dashboard and attachment storage are private.
- Confirm logs and batch history persist across worker restarts.

12. MVP versus later enhancements

MVP	Later enhancements
Single Gmail account	Multiple accounts and users
CSV and JSON	Google Sheets / Excel import
One shared CV PDF	Multiple per-recipient attachments
Plain-text messages	HTML editor and templates
Single persistent worker	Multiple workers and advanced queue management
Basic status/history	Analytics, bounce tracking and delivery reporting
Manual batch confirmation	Saved scheduling presets and quiet hours

Final recommendation

Build in this order: import and validation → Gmail OAuth and test-send → database-backed worker → scheduling and controls → crash recovery and deployment. Keep the UI simple. The main success criteria are that the worker runs while your laptop is off, each row maps to the correct individual email, and retries never blindly duplicate uncertain sends.

Implementation reference links: Gmail API send endpoint — https://developers.google.com/workspace/gmail/api/reference/rest/v1/users.messages/send ; Gmail API quotas — https://developers.google.com/workspace/gmail/api/reference/quota ; Google OAuth — https://developers.google.com/identity/protocols/oauth2 ; Supabase security — https://supabase.com/docs/guides/database/secure-data ; Supabase Storage access control — https://supabase.com/docs/guides/storage/security/access-control